Übungsblatt 4 – Lösungen

VC Logik

Aufgabe 1 (4 Punkte) – Sudoku-Formel

Angabe. Entwerfen Sie eine aussagenlogische Formel, die ein 4×4 Sudoku mit 2×2 Regionen repräsentiert (jedes Modell = eine Lösung). Jede Ziffer 1–4 soll in jeder Zeile, Spalte und Region höchstens einmal vorkommen.

Spielregeln.

- Das Spielfeld ist ein 4×4-Gitter mit 16 Feldern.
- Es ist in vier 2×2-Regionen unterteilt (links oben, rechts oben, links unten, rechts unten).
- Jedes Feld muss mit genau einer Ziffer aus $\{1, 2, 3, 4\}$ befüllt werden.
- In jeder **Zeile** darf jede Ziffer höchstens einmal vorkommen.
- In jeder **Spalte** darf jede Ziffer höchstens einmal vorkommen.
- In jeder **Region** darf jede Ziffer höchstens einmal vorkommen.

Da es 4 Ziffern und 4 Felder pro Zeile/Spalte/Region gibt, bedeutet "höchstens einmal" hier sogar "genau einmal".

Schritt 1: Variablen einführen. Für $i, j, n \in \{1, 2, 3, 4\}$ definiere

$$x_{i,j,n} \;:=\; \text{“Feld } (i,j) \text{ enthält Ziffer } n\text{”}.$$

Das sind $4 \cdot 4 \cdot 4 = 64$ Variablen.

Schritt 2: Regionen definieren. Die vier 2×2-Regionen:

$$\begin{aligned}
R_1 &= \{(1,1),(1,2),(2,1),(2,2)\} \quad \text{(links oben)}\\
R_2 &= \{(1,3),(1,4),(2,3),(2,4)\} \quad \text{(rechts oben)}\\
R_3 &= \{(3,1),(3,2),(4,1),(4,2)\} \quad \text{(links unten)}\\
R_4 &= \{(3,3),(3,4),(4,3),(4,4)\} \quad \text{(rechts unten)}
\end{aligned}$$

Schritt 3: Constraints formulieren. Fünf Bedingungen als Teilformeln; die Gesamtformel ist ihre Konjunktion.

(C1) Jedes Feld enthält mindestens eine Ziffer:

$$C_1 \;\equiv\; \bigwedge_{i=1}^{4} \bigwedge_{j=1}^{4} \big(x_{i,j,1} \vee x_{i,j,2} \vee x_{i,j,3} \vee x_{i,j,4}\big).$$

16 Klauseln (eine pro Feld).

(C2) Jedes Feld enthält höchstens eine Ziffer:

$$C_2 \;\equiv\; \bigwedge_{i=1}^{4} \bigwedge_{j=1}^{4} \bigwedge_{1 \le n < m \le 4} \neg\big(x_{i,j,n} \wedge x_{i,j,m}\big).$$

$16 \cdot 6 = 96$ Klauseln.

(C3) Keine Ziffer zweimal in derselben Zeile:

$$C_3 \;\equiv\; \bigwedge_{i=1}^{4} \bigwedge_{n=1}^{4} \bigwedge_{1\le j<k\le 4} \neg\big(x_{i,j,n} \wedge x_{i,k,n}\big).$$

$4 \cdot 4 \cdot 6 = 96$ Klauseln.

(C4) Keine Ziffer zweimal in derselben Spalte:

$$C_4 \;\equiv\; \bigwedge_{j=1}^{4} \bigwedge_{n=1}^{4} \bigwedge_{1\le i<k\le 4} \neg\big(x_{i,j,n} \wedge x_{k,j,n}\big).$$

$4 \cdot 4 \cdot 6 = 96$ Klauseln.

(C5) Keine Ziffer zweimal in derselben Region:

$$C_5 \;\equiv\; \bigwedge_{R\in\{R_1,R_2,R_3,R_4\}} \bigwedge_{n=1}^{4} \bigwedge_{\substack{(i,j),(k,l)\in R \\ (i,j)\neq(k,l)}} \neg\big(x_{i,j,n} \wedge x_{k,l,n}\big).$$

$4 \cdot 4 \cdot 6 = 96$ Klauseln.

Ergebnis Aufgabe 1.

$$\Phi \;\equiv\; C_1 \wedge C_2 \wedge C_3 \wedge C_4 \wedge C_5.$$

Insgesamt $16 + 96 + 96 + 96 + 96 = 400$ Klauseln über 64 Variablen. Jedes Modell von Φ entspricht einer gültigen Sudoku-Lösung.

Aufgabe 2 (2+2 Punkte) – Schaltkreis

Angabe. Stelle den gegebenen Schaltkreis als aussagenlogische Formel dar, deren Modelle genau die zulässigen Konfigurationen der Inputs/Outputs A, B, C, D, E, F sind. Notation: $[> k]$ gibt 1 aus, wenn die Input-Summe $> k$ ist; $[= k]$ gibt 1 aus, wenn die Input-Summe $= k$ ist.

Schritt 1: Verkabelung ablesen. Aus dem Schaltkreis liest man folgende Struktur ab (mit Hilfs-Outputs o_1, o_2, o_3 der linken Gates):

Gate	Inputs	Bedingung	Output
$[> 1]$ (oben links)	A, B	$A + B > 1$	o_1
$[= 1]$ (mitte links)	A, C	$A + C = 1$	o_2
$[> 0]$ (unten links)	B, C	$B + C > 0$	o_3
$[= 0]$ (oben rechts)	o_1	$o_1 = 0$	D
$[> 2]$ (mitte rechts)	o_1, o_2, o_3	Summe > 2	E
$[> 1]$ (unten rechts)	o_2, o_3	$o_2 + o_3 > 1$	F

Schritt 2: Gate-Bedingungen als Bool'sche Ausdrücke. Für binäre Inputs gilt:

$$\begin{aligned}
A + B > 1 &\Leftrightarrow A \wedge B \\
A + C = 1 &\Leftrightarrow (A \wedge \neg C) \vee (\neg A \wedge C) \\
B + C > 0 &\Leftrightarrow B \vee C \\
o_1 = 0 &\Leftrightarrow \neg o_1 \\
o_1 + o_2 + o_3 > 2 &\Leftrightarrow o_1 \wedge o_2 \wedge o_3 \\
o_2 + o_3 > 1 &\Leftrightarrow o_2 \wedge o_3
\end{aligned}$$

Schritt 3: Gesamtformel. Für jeden Draht/Output wird eine Äquivalenz geschrieben und alles mit $\wedge$ verknüpft:

$$\begin{aligned}
\Phi \equiv\ & (o_1 \leftrightarrow (A \wedge B)) \wedge \\
& (o_2 \leftrightarrow ((A \wedge \neg C) \vee (\neg A \wedge C))) \wedge \\
& (o_3 \leftrightarrow (B \vee C)) \wedge \\
& (D \leftrightarrow \neg o_1) \wedge \\
& (E \leftrightarrow (o_1 \wedge o_2 \wedge o_3)) \wedge \\
& (F \leftrightarrow (o_2 \wedge o_3)).
\end{aligned}$$

Die Modelle von Φ (projiziert auf A, B, C, D, E, F) sind genau die zulässigen Konfigurationen.

Anmerkung. Da o_1, o_2, o_3 Hilfsvariablen sind, lassen sie sich auch eliminieren und direkt durch die folgende äquivalente Formel ersetzen:

$$\begin{aligned}
\Phi' \equiv\ & (D \leftrightarrow \neg(A \wedge B)) \wedge \\
& (E \leftrightarrow ((A \wedge B) \wedge ((A \wedge \neg C) \vee (\neg A \wedge C)) \wedge (B \vee C))) \wedge \\
& (F \leftrightarrow (((A \wedge \neg C) \vee (\neg A \wedge C)) \wedge (B \vee C))).
\end{aligned}$$

Diese Form ist logisch äquivalent bezüglich der sichtbaren Variablen A, B, C, D, E, F.

Schritt 4: Teilaufgaben (a), (b), (c). Alle drei Fragen lassen sich mit einem Erfüllbarkeitstester auf Φ beantworten:

	Frage	**Dem Tester geben**
(a)	Kann $F = 1$ sein?	$\Phi \wedge F$ erfüllbar $\Rightarrow$ ja
(b)	Kann $E = A$ sein?	$\Phi \wedge (E \leftrightarrow A)$ erfüllbar $\Rightarrow$ ja
(c)	Gilt $F = 1 \Leftrightarrow D = 0$?	$\Phi \wedge \neg(F \leftrightarrow \neg D)$ unerfüllbar $\Rightarrow$ ja

Konkrete Antworten (durch explizite Belegungen):

(a) Ja, $F = 1$ ist möglich.
Wähle $A = 1, B = 1, C = 0$. Dann: $o_2 = (A \wedge \neg C) \vee (\neg A \wedge C) = (1 \wedge 1) \vee (0 \wedge 0) = 1$, $o_3 = B \vee C = 1 \vee 0 = 1$, $F = o_2 \wedge o_3 = 1 \wedge 1 = 1$. ✓

(b) Ja, $E = A$ ist möglich.
Trivial für $A = 0$: wähle $A = B = C = 0$, dann $E = 0 \wedge \ldots = 0 = A$. ✓
Auch für $A = 1$: wähle $A = 1, B = 1, C = 0$. Dann $o_1 = 1$, $o_2 = 1$, $o_3 = 1$, also $E = 1 = A$. ✓

(c) Nein, $F = 1 \Leftrightarrow D = 0$ gilt NICHT.
Gegenbeispiel: $A = B = C = 1$. Dann: $o_1 = A \wedge B = 1$, also $D = \neg o_1 = 0$. $o_2 = (A \wedge \neg C) \vee (\neg A \wedge C) = 0$, $o_3 = B \vee C = 1$, $F = o_2 \wedge o_3 = 0$. Hier ist $D = 0$, aber $F = 0 \neq 1$. Also nicht äquivalent.

Aufgabe 3 (4 Punkte) – Bishop Independence Problem

Angabe. Schreiben Sie ein Skript, das Formeln produziert, die das Bishop Independence Problem lösen (analog zu Rook/Queen Independence Problem).

Schritt 1: Problem verstehen. Auf einem $n \times n$ Schachbrett: Platziere Läufer so, dass keine zwei einander schlagen können. Läufer bewegen sich diagonal, d. h. zwei Läufer schlagen sich genau dann, wenn sie auf derselben Diagonale stehen.

Diagonalen:

- Hauptdiagonalen (von links oben nach rechts unten): $i - j$ ist konstant, $i - j \in \{-(n-1), \ldots, n-1\}$ – insgesamt $2n - 1$ Stück.
- Gegendiagonalen (von rechts oben nach links unten): $i + j$ ist konstant, $i + j \in \{2, \ldots, 2n\}$ – ebenfalls $2n - 1$ Stück.

Schritt 2: Variablen und Constraints. Für jedes Feld (i, j) mit $1 \leq i, j \leq n$ definiere

$$b_{i,j} \ := \text{“Auf Feld } (i,j) \text{ steht ein Läufer”}.$$

Für jede Diagonale D (mit mindestens zwei Feldern) und jedes Paar $(i_1, j_1), (i_2, j_2) \in D$ mit $(i_1, j_1) \neq (i_2, j_2)$:

$$\neg\big(b_{i_1,j_1} \wedge b_{i_2,j_2}\big).$$

Die Gesamtformel ist die Konjunktion all dieser Klauseln.

Schritt 3: Python-Skript.

```python
def bishop_independence(n):
    """
    Erzeugt die aussagenlogische Formel (als Liste von Klauseln)
    fuer das Bishop Independence Problem auf einem n x n
    Schachbrett. Jede Klausel hat die Form
    (not b_{i1,j1} or not b_{i2,j2}) und verbietet zwei Laeufer
    auf derselben Diagonale.
    """
    # Alle Diagonalen sammeln
    diagonals = {}
    for i in range(1, n+1):
        for j in range(1, n+1):
            # Hauptdiagonale: i - j ist konstant
            diagonals.setdefault(('haupt', i-j), []).append((i, j))
            # Gegendiagonale: i + j ist konstant
            diagonals.setdefault(('gegen', i+j), []).append((i, j))

    # Klauseln erzeugen: fuer jede Diagonale mit >= 2 Feldern,
    # fuer jedes Paar ein at-most-one-Constraint
    clauses = []
    for feld_liste in diagonals.values():
        if len(feld_liste) < 2:
            continue
        for p in range(len(feld_liste)):
            for q in range(p+1, len(feld_liste)):
                (i1, j1) = feld_liste[p]
                (i2, j2) = feld_liste[q]
```

```python
                klausel = f"(NOT b_{{{i1},{j1}}} OR NOT b_{{{i2},{j2}}})"
                clauses.append(klausel)
    return clauses


def formel_ausgabe(clauses):
    """Verbindet alle Klauseln mit AND zur Gesamtformel."""
    return " AND ".join(clauses)


if __name__ == "__main__":
    n = 4
    klauseln = bishop_independence(n)
    print(f"Anzahl Klauseln fuer n={n}: {len(klauseln)}")
    print(formel_ausgabe(klauseln))
```

Schritt 4: Beispiel $n = 4$**.** Für ein 4×4-Brett:

- Hauptdiagonalen: 7 (für $i - j \in \{-3, -2, -1, 0, 1, 2, 3\}$). Größen: $1, 2, 3, 4, 3, 2, 1$.
- Gegendiagonalen: 7 (für $i + j \in \{2, 3, 4, 5, 6, 7, 8\}$). Größen: $1, 2, 3, 4, 3, 2, 1$.

Anzahl der Paare pro Diagonale: $\binom{k}{2}$ für Größe k. Also $\binom{2}{2} + \binom{3}{2} + \binom{4}{2} + \binom{3}{2} + \binom{2}{2} = 1 + 3 + 6 + 3 + 1 = 14$ pro Richtung. Insgesamt $14 + 14 = 28$ Klauseln.

Ergebnis Aufgabe 3.

Das Skript produziert für jedes n eine Formel in CNF, die nur die Independence-Constraints enthält. Modelle = gültige Läufer-Konfigurationen (ohne gegenseitiges Schlagen). Triviale Modelle (leeres Brett) sind eingeschlossen; wird eine Mindestanzahl an Läufern gefordert, muss das als zusätzlicher Constraint ergänzt werden.